VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5
Max Marks: 100

Student Name	Registration Number
Goutam Bhawsar	24BCG10087
Slot	**Date of Submission**
D11	31/03/2026

1. Introduction

Most of the software that runs the modern world from the servers that run websites to the tools developers use every day is not under a single entity's control. Instead, it is developed openly, shared freely and enhanced by a global community. This concept is the basis for opensource software. One of the most important tools within this ecosystem is Git, a distributed version control system which has become a must-have for modern software development. Created by Linus Torvalds in 2005, Git helps developers to trace changes, collaborate effectively, and keep the history of complete projects in control without having to depend on a central authority. Today, it is used as the backbone of platforms such as GitHub and GitLab, and is used to support millions of projects across the world. This project will be about making thing like a Structure audit of Git. The goal isn't just to know what Git does, but also what the problem was that Git solved and what Philosophy values are there behind. The report discusses where it came from, the licensing model for it, the ethical implications, its role in the Linux systems, and how the larger opensource ecosystem is affected by it. In addition of the theoretical part, this project contains 5 shell scripts, which show towards the practical interaction with Linux systems. These scripts are targeted to mirror the actual using opensource tools via the command line as a factor of automation and transparency is a central part of opensource concept. It is important to understand such tools as git because modern software development is developed using opensource foundations. As future professionals, it is important to not only use these tools, but also to know the ideas and principles behind their use.

2. Choosing Your Software

Pick one open-source project from the list below, or propose your own (subject to instructor approval). Your choice will be the subject of your entire report and all five shell scripts.

Approved software options

Software	Category	License	Why it's interesting
Linux Kernel	Operating System	GPL v2	*The foundation everything else runs on*
Apache HTTP Server	Web Server	Apache 2.0	*Powers ~30% of all websites globally*
MySQL	Database	GPL v2 / Commercial	*A license with a dual-model story to tell*
Firefox	Web Browser	MPL 2.0	*A nonprofit fighting for an open web*
VLC Media Player	Multimedia	LGPL / GPL	*Plays anything — built by students in Paris*
LibreOffice	Office Suite	MPL 2.0	*Born from a community fork — a real lesson*
Python	Programming Language	PSF License	*A language shaped entirely by community*
Git	Version Control	GPL v2	*The tool Linus built when proprietary failed him*

Your choice	Write your chosen software here before submission: **Git**

3. Project Report Structure

Part A — Origin and Philosophy (Units 1 & 2)

A1. The problem this software was created to solve

Before the advent of Git, working with large software, and working with a project that had many contributors, was a major pain. The Linux kernel, one of the world's largest collaborative opensource projects, was originally managed via traditional version control systems that were too slow, inefficient or reliant on centralized control. For a while the Linux development community used a proprietary program called BitKeeper. While BitKeeper was powerful and efficient, it had a big downside: It wasn't fully open source. This brought about a conflict within the Linux community which had a strong belief in openness and freedom. In 2005 the situation came to a head when BitKeeper revoked the free usage license for Linux developers over their disputes resulting from attempts at reverse engineering. Suddenly the entire Linux kernel team did not have access to their own most important work tool. This created an urgent need for a replacement which could be powerful, while completely open. Instead of using another one that is already out there, Linus Torvalds decided to create a new version control system based from the ground up. His goal was to invent something fast, distributed and independent of any central authority. Within a few weeks, a new childhood called Git was built with an emphasis on speed, data integrity and decentralized collaboration. Git solved several problems at once, removing dependency on proprietary tools, improving the speed of large-scale projects, and allowing every developer to have a complete copy of the repository. This made collaboration more workable and robust. The origin of Git reveals that it wasn't developed as a tool but as a reaction to a real problem where the control over the software threatened the freedom of developers. It's a reflection of the fundamental philosophy of open source - to construct systems that empower users rather than constrain them.

A2. The license — what it actually says

Git is distributed under the version 2 version of the GNU General Public License (GPL v2), which is one of the most widely used open source licenses. This license aims at establishing that the software is free, both in terms of use and terms of modification. The GPL license builds on four core freedoms, the four classic freedoms: the freedom to run the program for any purpose, the freedom to study how it works, the freedom to modify it, and the freedom to distribute copies. Git meets all four of these freedoms quite well, and is a true example of free software. One key aspect of the GPL license is that it's a "copyleft" license. This means that if a company modifies Git, and distributes the modified version of the software, they are required to also release the source code of the modifications. This helps to prevent companies from taking open source software and enhancing it privately and then restricting access to others. This is distinct from permissive licenses, such as the MIT License. The MIT License is a type of license that gives developers the freedom to modify and distribute software without the requirement to share their modifications. While this provides more flexibility with commercial use, it can also mean situations where improvements are not returned to the community. If I was going to do my own software, one choice would be the GPL if my goal was to have long term openness and contribution from a community. However, if I wanted maximum adoption and flexibility, then I may want to consider the MIT license instead. Another useful distinction to make is between "free as in free beer," and "free as in freedom." Git is not just free to download and use, but it freedoms users to understand, modify, and share the software! This makes it considerably more valuable than software that's free just in the sense of cost.

A3. The ethics of open source

Open source is not just about openness to software, but is also a philosophy of how knowledge and technology should be shared. It poses some interesting ethical questions about ownership, collaboration and fairness in the software industry. One of the important questions to consider is whether all software should be open source. On one hand, communal knowledge and, in terms of software, open-source software ensures transparency and security, and spurs innovation. And when source code is available, just anyone can look through it for bugs or security flaws, and improvements can come from a global community. On the other hand, not all software can be open source, with intellectual property or security considerations being important industries, for example. Companies also require financial incentives in order to continue development. Another ethical issue is whether it's fair that companies are building profitable products on top of open source software. Many companies like those using Linux-based system are generating revenue with tools developed by community contributions. This can be considered ethical if the company contributes back to the community through code, funding, and/or support. However, if a company is only benefiting, without contributing, it does raise concerns about exploitation. The phrase "standing on the shoulders of giants" refers to the idea that innovation stands on the shoulders of what's gone on before. In the context of open source this idea is all the more powerful. Developers can develop new technologies without starting from scratch, which helps to expedite progress. Rather than inhibition of innovation, open source actually promotes it, by grinding down the barrier to entry. In my opinion, open source helps ensure a balance between contributing as an individual and advancing towards everyone's benefit. The way this is done is by enabling innovation to flourish and grow more rapidly without losing the ability to make the knowledge available to everyone.

Part B — Linux Footprint (Unit 2)

This section examines how Git exists and operates within a Linux system. Unlike traditional applications that run as background services, Git is a command-line tool that interacts directly with the user and the file system. Its design reflects the philosophy of simplicity and flexibility in Linux-based environments.

Installation Method

Git can be easily installed on a Linux using the package manager. On Debian-based systems it is available through the Advanced Package Tool. The installation can be performed using the following command:

```
sudo apt update
sudo apt install git
```

Once installed, the version can be verified using:

```
git version
```

This shows that Git is maintained within the official package repositories, making it easy to install and update without manual compilation.

Key Directories

Moreover, after installation, Git does not have a large directory structure like server applications. Its components are distributed across standard Linux directories.

- /usr/bin/git — Contains the main executable file
- /etc/gitconfig — System-wide configuration file
- ~/.gitconfig — User-specific configuration file
- .git/ — Repository-specific metadata directory (created when a repository is initialized)

The .git directory is especially important because it holds all version history, commits, and configurations related to a project. This again proves that Git is designed to store project data locally and separately.

User and Group

Git does not run as a service or a daemon. Instead, it runs in the context of the user currently logged in. This means that all actions performed by Git, such as commits, cloning, and branching, run at this user's permissions.

This is a security feature, as Git does not run at a heightened privilege level. Users can also perform all actions on their own repositories without affecting anything at a system level.

Service Management

Unlike other software like Apache and MySQL, Git is not run in the background. This means that Git is not run through systemctl commands like start, stop, and restart.
Instead, Git is executed directly through the command line whenever needed. For example:

git init → Initializes a new repository
git status → Shows current changes
git log → Displays commit history

This on-demand nature of Git minimizes resource consumption and is in line with the Unix philosophy of executing applications only when necessary.

Update Model

Git is updated through the Linux package manager. When the system is updated using commands such as:

sudo apt update
sudo apt upgrade

the latest stable version of Git is installed along with any security patches.

As Git is an open-source project, updates and bug fixes are done by people all over the world. They are then tested and released through official repositories.

This way, users can get updates and patches without relying on any single entity.

Practical Verification

To confirm Git's presence on the system, the following commands can be used:

which git
git --version
ls -l /usr/bin/git

These commands assist in the confirmation of the installation, version, and file permissions, which provide a clear example of how the program interacts with the Linux file system.

In conclusion, it is evident that the Git program is a natural fit for the Linux operating system, as it is light, uses the command line, and is a decentralized system. It is also efficient, as it does not run in the background, and it is safe, as it is powerful.

Part C — The FOSS Ecosystem (Units 3 & 4)

No opensource software is ever created in isolation. Rather, every opensource software depends on other opensource software and also provides support for other opensource software. Git is a great example of this interconnected ecosystem of opensource tools and platforms.

Dependencies of Git

Git itself is written primarily in the C programming language and relies on standard Linux system libraries. It does not require heavy external dependencies, which makes it lightweight and efficient. However, it works closely with other tools such as:

• GCC (GNU Compiler Collection) — used to compile Git from source
• OpenSSL — for secure data transfer in remote repositories
• libcurl — for handling HTTP/HTTPS communication
• zlib — for data compression and efficient storage

What Git Has Enabled

Git has played a major role in shaping the modern software development ecosystem. It has enabled the creation of platforms such as:

• GitHub — a web-based hosting platform for Git repositories
• GitLab — an integrated DevOps platform
• Bitbucket — another Git-based collaboration platform

These platforms provide features like version control, issue tracking, collaboration, and continuous integration. Without Git, many of these tools would not exist in their current form.

Git has also inspired tools and workflows such as:
• Distributed version control systems
• Branching and merging workflows (Git Flow)
• Continuous integration and deployment pipelines.

Role in the Software Stack

Git does not directly belong to the traditional stack (Linux, Apache, MySQL, PHP), but it plays a supporting role in modern web development.

Developers use Git to manage the source code of applications that run on LAMP-based systems. For example:
Code written in PHP is tracked using Git
Web server configurations are version-controlled
Database scripts are maintained and updated using Git repositories.

Community and Governance

The maintenance of Git is handled by a global community of developers, and Linus Torvalds continues to play a key role in the development process.

The Git community communicates through:
Mailing lists
Online forums
Git repositories and issue trackers

Unlike other software, Git is not owned or controlled by a single entity or company. Its development is managed through contributions to the community and peer review.

Ecosystem Impact

Git has revolutionized how developers collaborate. Git has enabled distributed development, whereby developers in various parts of the world can contribute to a single project.

Git has also made it easier for new developers to come into the field because they can easily access open-source projects. This has enabled rapid growth in innovation and knowledge sharing in the software field.

Git is not just a simple tool; it is an integral part of open-source technology. Git connects developers, platforms, and technologies; therefore, it is one of the most influential software projects in modern computing history.

Part D — Open Source vs Proprietary

To understand the practical value of Git, it is useful to compare it with a traditional version control system such as SVN (Subversion), which follows a more centralized and controlled approach.

Dimension	Open Source	Proprietary Alternative
Cost	Free and open source	Often requires enterprise support or paid hosting
Security (who can audit the code?)	Code is open to audit by anyone, increasing transparency	Limited visibility; depends on organization for security checks
Support and reliability	Community-driven support, extensive documentation	Official/vendor support available but limited to provider
Freedom to modify	Full freedom to modify and redistribute under GPL license	Limited flexibility; depends on licensing and system control
Community vs corporate control	Community-driven development and governance	Centralized control under a specific organization
Your overall verdict	Distributed (every user has full repository)	Centralized (single main server)

Overall Verdict

Based on this comparison, Git offers significantly more flexibility and control to developers compared to centralized systems like SVN. Its distributed nature ensures that every user has a complete copy of the repository, which improves reliability and enables offline work. Additionally, the open-source model allows developers to inspect, modify, and contribute to the software, making it more adaptable to changing needs.

However, centralized systems like SVN may still be useful in environments where strict control and simpler workflows are required, especially for beginners or organizations that prefer a single point of management. Therefore, the choice depends on the specific use case, but for modern software development, Git is generally more suitable.

Conclusion

Open source is not just about openness to software, but is also a philosophy of how knowledge and technology should be shared. One of the important questions to consider is whether all software should be open source. And when source code is available, just anyone can look through it for bugs or security flaws, and improvements can come from a global community. On the other hand, not all software can be open source: some industries do not even allow open source software due to intellectual property or security concerns. Companies also require financial incentives to continue development. Another ethical issue is whether it's fair that companies are building profitable products on top of open source software. Many companies like those using Linux-based system are generating revenue with tools developed by community contributions. This can be considered ethical if the company contributes back to the community

through code, funding, and/or support. However, if a company is only benefiting, without contributing, it does raise concerns about exploitation. "Standing on the shoulders of giants" describes the innovation that rests on the shoulders of that which came before. In the context of open source this idea is all the more powerful. Developers can develop new technologies without starting from scratch, which helps to expedite progress. Rather than making it harder for innovation to occur, open source in fact adds to innovation by lowering the barriers of entry. In my mind, open source allows for a balance between a alone and many together. The way this is done is by enabling innovation to flourish and grow more rapidly without losing the ability to make the knowledge available to everyone.

4. Shell Script Tasks

Script 1 — System Identity Report (Unit 1 & 2)

Open source is not just about openness to software, but is also a philosophy of how knowledge and technology should be shared. It poses some important ethical questions regarding ownership, collaboration, and fairness in the software industry. One of the important questions to consider is whether all software should be open source. On the one hand, opensource software fosters transparency, security and innovation. And when source code is available, just anyone can look through it for bugs or security flaws, and improvements can come from a global community. On the other hand, not all software is eligible for open-source software, particularly in industries where intellectual property or security are of essence. Companies also need financial incentives to continue with development. Another ethical issue is whether it's fair that companies are building profitable products on top of opensource software. Many companies like those using Linux-based system are generating revenue with tools developed by community contributions. This can be considered ethical if the company contributes back to the community through code, funding, and/or support. However, if a company is only benefiting, without contributing, it does raise concerns about exploitation. The that simile means that innovation is based on previous work. In the context of open source this idea is all the more powerful. Developers can develop new technologies without starting from scratch, which helps to expedite progress. Rather than slowing innovation down, open source does just the opposite by removing barriers to entry. In my opinion, open source is how we strike a balance between individual contribution and group progress. The way this is done is by enabling innovation to flourish and grow more rapidly without losing the ability to make the knowledge available to everyone.

CODE:-

```
#!/bin/bash

# Script 1: System Identity Report
# Author: Goutam Bhawsar
# Course: Open Source Software

# --- Variables ---
STUDENT_NAME="Goutam Bhawsar"
SOFTWARE_CHOICE="Git"

# --- System Info ---
KERNEL=$(uname -r)
USER_NAME=$(whoami)
HOME_DIR=$HOME
UPTIME=$(uptime -p)
DATE_TIME=$(date)
DISTRO=$(grep '^PRETTY_NAME' /etc/os-release | cut -d= -f2 | tr -d '"')
GIT_VERSION=$(git --version 2>/dev/null)
```

```
# --- Display ---
echo "========================================"
echo " Open Source Audit — $STUDENT_NAME"
echo "========================================"
echo "Software Chosen : $SOFTWARE_CHOICE"
echo "Distribution    : $DISTRO"
echo "Kernel Version  : $KERNEL"
echo "User            : $USER_NAME"
echo "Home Directory  : $HOME_DIR"
echo "Uptime          : $UPTIME"
echo "Date & Time     : $DATE_TIME"
echo "Git Version     : ${GIT_VERSION:-'Git not installed'}"
echo ""
echo "License Note    : Linux kernel is licensed under GPL v2"
echo "========================================"
```

OUTPUT:-

```
root@Gautam: /mnt/c/Users/
root@Gautam:~# cd /mnt/c/Users/gauta/OneDrive/Desktop/oss-audit-24BCG10087/scripts
root@Gautam:/mnt/c/Users/gauta/OneDrive/Desktop/oss-audit-24BCG10087/scripts# ls
script1_system_identity.sh    script3_disk_auditor.sh  script5_manifesto.sh
script2_package_inspector.sh  script4_log_analyzer.sh
root@Gautam:/mnt/c/Users/gauta/OneDrive/Desktop/oss-audit-24BCG10087/scripts# chmod +x *.sh
./script1_system_identity.sh
========================================
 Open Source Audit — Goutam Bhawsar
========================================
Software Chosen : Git
Distribution    : Ubuntu 24.04.1 LTS
Kernel Version  : 5.15.167.4-microsoft-standard-WSL2
User            : root
Home Directory  : /root
Uptime          : up 11 minutes
Date & Time     : Mon Mar 30 16:20:59 UTC 2026
Git Version     : git version 2.43.0

License Note    : Linux kernel is licensed under GPL v2
========================================
```

Script 2 — FOSS Package Inspector (Unit 2)

Open source is not just about openness to software, but is also a philosophy of how knowledge and technology should be shared. It raises important ethical questions about ownership, collaboration and fairness in the software industry. One of the important questions to consider is whether all software should be open source. On one hand, open-source software encourages transparency, security, and innovation. And when source code is available, just anyone can look through it for bugs or security flaws, and improvements can come from a global community. On the other hand, not all software can be open source, particularly in industries where intellectual property or security concerns are of paramount importance. Companies also require financial incentives to maintain development. Another ethical issue is whether it's fair that companies are building profitable products on top of open source software. Many companies like those using Linux-based system are generating revenue with tools developed by community contributions. This can be considered ethical if the company contributes back to the community through code, funding, and/or support. The meaning of the phrase "standing on the shoulders of giants" is that innovation builds on previous work. In the context of open source this idea is all the more powerful. Developers can develop new technologies without starting from scratch, which helps to expedite progress. Rather than having a dampening effect on innovation, open source actually

fosters innovation by lowering barriers-to-entry. In my opinion, open source forms a kind of equal state between individual contribution and the changes of one collective. The way this is done is by enabling innovation to flourish and grow more rapidly without losing the ability to make the knowledge available to everyone.

CODE:-

```
#!/bin/bash

# Script 2: FOSS Package Inspector

PACKAGE="git"

echo "Checking package: $PACKAGE"
echo "--------------------------------"

# Check installation (Debian-based systems)
if dpkg -l | grep -qw $PACKAGE; then
  echo "$PACKAGE is installed."

  # Show details
  dpkg -s $PACKAGE | grep -E 'Version|Maintainer|Description'
else
  echo "$PACKAGE is NOT installed."
fi

echo ""
echo "Philosophy Note:"
echo "--------------------------------"

# Case statement for package meaning
case $PACKAGE in
  git) echo "Git: distributed version control that empowers open collaboration"
;;
  apache2) echo "Apache: the web server that built the open internet" ;;
  mysql) echo "MySQL: open source at the heart of millions of applications" ;;
  firefox) echo "Firefox: a browser promoting an open and private web" ;;
  *) echo "Unknown package — still part of the open-source ecosystem" ;;
esac
```

OUTPUT:-

```
root@Gautam:/mnt/c/Users/gauta/OneDrive/Desktop/oss-audit-24BCG10087/scripts# ./script2_package_inspector.sh
Checking package: git
--------------------------------
git is installed.
Maintainer: Ubuntu Developers <ubuntu-devel-discuss@lists.ubuntu.com>
Version: 1:2.43.0-1ubuntu7.1
Description: fast, scalable, distributed revision control system
Original-Maintainer: Jonathan Nieder <jrnieder@gmail.com>

Philosophy Note:
--------------------------------
Git: distributed version control that empowers open collaboration
```

Script 3 — Disk and Permission Auditor (Unit 2)

Open source is not just about openness to software, but is also a philosophy of how knowledge and technology should be shared. It opens up important ethical questions of ownership, collaboration, and fairness in the software industry. One of the important questions to consider is whether all software should be open source. On one hand, the principles of open-source software encourage transparency and security and foster innovation. And when source code is available, just anyone can look through it for bugs or security flaws, and improvements can come from a global community. On the other hand, not all software can be open source, particularly in industries where intellectual property or security concerns are very important. Companies also need a financial incentive to continue development. Another ethical issue is whether it's fair that companies are building profitable products on top of open source software. Many companies like those using Linux-based system are generating revenue with tools developed by community contributions. This can be considered ethical if the company contributes back to the community through code, funding, and/or support. However, if a company is only benefiting, without contributing, it does raise concerns about exploitation. The term 'standing on the shoulders of giants' means building innovation on the work that has come before you. In the context of open source this idea is all the more powerful. Developers can develop new technologies without starting from scratch, which helps to expedite progress. Rather than being a drag on innovation, open source actually promotes it by lowering barriers-to-entry. In my view, open source forms a peace between individual and common course. The way this is done is by enabling innovation to flourish and grow more rapidly without losing the ability to make the knowledge available to everyone.

CODE:-

```
#!/bin/bash

# Script 3: Disk and Permission Auditor

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "Directory Audit Report"
echo "------------------------------"

for DIR in "${DIRS[@]}"; do
  if [ -d "$DIR" ]; then
    PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
    SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)
    echo "$DIR => Permissions: $PERMS | Size: $SIZE"
  else
    echo "$DIR does not exist"
  fi
done

echo ""
echo "Git Configuration Check"
echo "------------------------------"
```

```
GIT_CONFIG="$HOME/.gitconfig"

if [ -f "$GIT_CONFIG" ]; then
  echo "Git config file found:"
  ls -l $GIT_CONFIG
else
  echo "Git config file not found."
fi
```

OUTPUT:-

```
root@Gautam:/mnt/c/Users/gauta/OneDrive/Desktop/oss-audit-24BCG10087/scripts# git config --global user.name "gb1325"
git config --global user.email "gautambhawsar13@gmail.com"
root@Gautam:/mnt/c/Users/gauta/OneDrive/Desktop/oss-audit-24BCG10087/scripts# ls -a ~
.  ..  .bash_history  .bashrc  .cache  .gitconfig  .motd_shown  .profile  .ssh
root@Gautam:/mnt/c/Users/gauta/OneDrive/Desktop/oss-audit-24BCG10087/scripts# ./script3_disk_auditor.sh
Directory Audit Report
--------------------------------
/etc => Permissions: drwxr-xr-x root root | Size: 4.2M
/var/log => Permissions: drwxrwxr-x root syslog | Size: 25M
/home => Permissions: drwxr-xr-x root root | Size: 4.0K
/usr/bin => Permissions: drwxr-xr-x root root | Size: 92M
/tmp => Permissions: drwxrwxrwt root root | Size: 52K

Git Configuration Check
--------------------------------
Git config file found:
-rw-r--r-- 1 root root 57 Mar 30 16:40 /root/.gitconfig
```

Script 4 — Log File Analyzer (Units 2 & 5)

Open source is not just about openness to software, but is also a philosophy of how knowledge and technology should be shared. It raises important questions of ethics about ownership, collaboration and fairness in the software industry. One of the important questions to consider is whether all software should be open source. On one hand, opensource software encourages transparency, security, and innovation. And when source code is available, just anyone can look through it for bugs or security flaws, and improvements can come from a global community. On the other hand, not all software can be open source, particularly in industries where intellectual property or security issues are of paramount importance. Companies also require financial incentives in order to continue with development. Another ethical issue is whether it's fair that companies are building profitable products on top of opensource software. Many companies like those using Linux-based system are generating revenue with tools developed by community contributions. This can be considered ethical if the company contributes back to the community through code, funding, and/or support. However, if a company is only benefiting, without contributing, it does raise concerns about exploitation. The meaning of "standing on the shoulders of giants" is that you build your innovations on the work of past. In the context of open source this idea is all the more powerful. Developers can develop new technologies without starting from scratch, which helps to expedite progress. Rather, open source could have increased innovation by eliminating barriers to entry. In my opinion open source helps attain a balance between individual contribution and collective progress. The way this is done is by enabling

innovation to flourish and grow more rapidly without losing the ability to make the knowledge available to everyone.

CODE:-

```
#!/bin/bash

# Script 4: Log File Analyzer
# Usage: ./script4_log_analyzer.sh <logfile> [keyword]

LOGFILE=$1
KEYWORD=${2:-"error"}
COUNT=0

# Check if file exists
if [ ! -f "$LOGFILE" ]; then
  echo "Error: File $LOGFILE not found."
  exit 1
fi

# Check if file is empty
if [ ! -s "$LOGFILE" ]; then
  echo "Warning: File is empty. Please provide a valid log file."
  exit 1
fi

# Read file line by line
while IFS= read -r LINE; do
  if echo "$LINE" | grep -iq "$KEYWORD"; then
    COUNT=$((COUNT + 1))
  fi
done < "$LOGFILE"

echo "--------------------------------"
echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"
echo "--------------------------------"

echo ""
echo "Last 5 matching lines:"
grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

OUTPUT:-

```
root@Gautam:/mnt/c/Users/gauta/OneDrive/Desktop/oss-audit-24BCG10087/scripts# ls /var/log
README           apt       bootstrap.log  dist-upgrade  dmesg.0     dpkg.log  fontconfig.log  kern.log   lastlog  syslog              wtmp
alternatives.log  auth.log  btmp           dmesg         dmesg.1.gz  faillog   journal         landscape  private  unattended-upgrades
root@Gautam:/mnt/c/Users/gauta/OneDrive/Desktop/oss-audit-24BCG10087/scripts# ./script4_log_analyzer.sh /var/log/dpkg.log error
--------------------------------
Keyword 'error' found 21 times in /var/log/dpkg.log
--------------------------------

Last 5 matching lines:
2025-01-06 20:15:02 status unpacked liberror-perl:all 0.17029-2
2025-01-06 20:15:30 configure liberror-perl:all 0.17029-2 <none>
2025-01-06 20:15:30 status unpacked liberror-perl:all 0.17029-2
2025-01-06 20:15:30 status half-configured liberror-perl:all 0.17029-2
2025-01-06 20:15:30 status installed liberror-perl:all 0.17029-2
```

Script 5 — The Open Source Manifesto Generator (Unit 5)

This script helps users create a personalized open-source philosophy statement. It asks users to provide answers to three questions about their experiences with open-source software and their understanding of freedom and sharing. The script uses this information to build a paragraph and saves it to a text file.

The script uses the read command to get user input and stores this input in variables. Then, it uses string concatenation and the echo command to build a paragraph. This paragraph is written to a file using the output redirection operator (>), and the date command adds the current date to the paragraph.

This script shows the power of shell scripting by using it creatively to solve a problem. While shell scripting can be used for system tasks, this script shows its capabilities extend beyond this.

CODE:-

```
#!/bin/bash

# Script 5: Open Source Manifesto Generator

echo "========================================"
echo " Open Source Manifesto Generator"
echo "========================================"
echo ""

# User input
read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD

DATE=$(date '+%d %B %Y')
USER_NAME=$(whoami)
OUTPUT="manifesto_${USER_NAME}.txt"

# Generate manifesto
echo "----------------------------------------" > $OUTPUT
echo "Open Source Manifesto" >> $OUTPUT
echo "Date: $DATE" >> $OUTPUT
echo "----------------------------------------" >> $OUTPUT
echo "" >> $OUTPUT

echo "On $DATE, I reflect on the importance of open source." >> $OUTPUT
echo "I use $TOOL regularly, and it represents the spirit of collaboration." >>
$OUTPUT
echo "To me, freedom means $FREEDOM — the ability to learn, modify, and share."
>> $OUTPUT
echo "I believe in building $BUILD and making it accessible to everyone." >>
$OUTPUT
```

```
echo "Open source is not just code, it is a philosophy of transparency and
community." >> $OUTPUT

echo ""
echo "Manifesto saved to $OUTPUT"
echo ""
cat $OUTPUT
```

OUTPUT:-

```
root@Gautam:/mnt/c/Users/gauta/OneDrive/Desktop/oss-audit-24BCG10087/scripts# ./script5_manifesto.sh
========================================
 Open Source Manifesto Generator
========================================

1. Name one open-source tool you use every day: Blender
2. In one word, what does 'freedom' mean to you? Doing everything i want upto my limits
3. Name one thing you would build and share freely: My Git Repositry

Manifesto saved to manifesto_root.txt

----------------------------------------
Open Source Manifesto
Date: 30 March 2026
----------------------------------------

On 30 March 2026, I reflect on the importance of open source.
I use Blender regularly, and it represents the spirit of collaboration.
To me, freedom means Doing everything i want upto my limits — the ability to learn, modify, and share.
I believe in building My Git Repositry and making it accessible to everyone.
Open source is not just code, it is a philosophy of transparency and community.
```